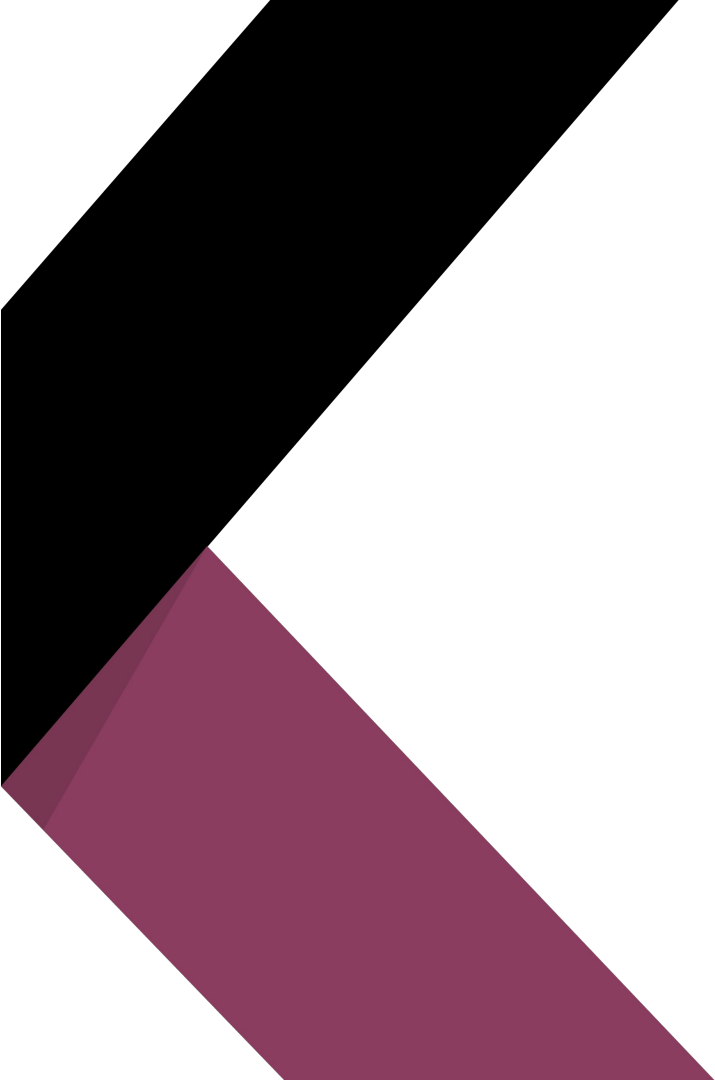




Diagnosis



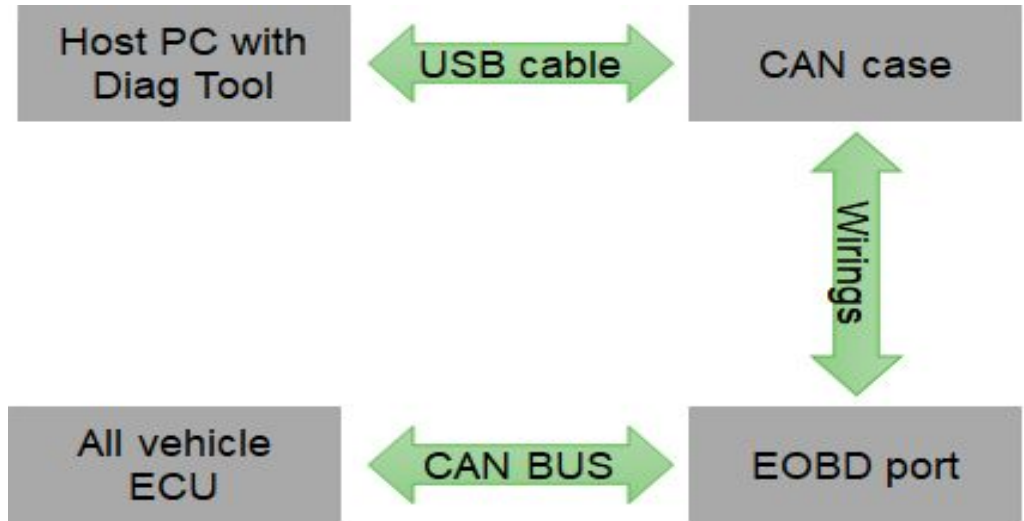
Introduction



Diagnostic Communication

The main features of diagnostic communication are:

- the communication channel is the CAN BUS;
- the diagnostic tool is an added node on the CAN network;
- the diagnostic tool requires for the diagnostic information to each ECU;
- the interested ECU provides the responses about the question;
- the translation of the diagnostic information from raw value to “engineering” value is performed by specific tools.





ISO Reference

- ISO 14229-1: Diagnostic services – Specification and requirements
- ISO 14229-3: Road vehicles — Unified diagnostic services (UDS) — Part 3: Unified diagnostic services on CAN implementation (UDSonCAN)
- ISO 15765: Road vehicles – Diagnostic on Controller Area Network (CAN)



Diagnostic – Definitions

- **Diagnosis:** It is the capacity of an electronic system (Control unit), equipped with microprocessor, to carry out a cyclic check, through the acquisition, processing and comparison of component signals directly or indirectly connected, of both internal (microprocessor, memory, etc.) and external (sensors and actuators) components belonging to the same system. The results of the diagnosis, in case of failure (permanent or intermittent) detection through the control unit, must be recorded in the dedicated error cells of the non-volatile memory.
- **Communication protocol:** It is the set of rules controlling and allowing the communication between one or more electronic systems and diagnostic instrument
- **Diagnostic instrument:** Electronic equipment able to guarantee the communication to one or more electronic systems provided with diagnosis.
- **Error cell:** It is a RAM or EEPROM memory cell containing the information (error code, environmental parameters) relevant to a single trouble



Diagnostic Performance

All control systems must foresee the following functions:

- managing a physical transmission ;
- recognizing the emergencies on all sensors and actuators directly connected to the system, on the power supply lines both entering and exiting the system and, if possible, identifying also mechanical part faults and malfunctions through a plausibility diagnosis;
- the self-diagnosis functions must be able to distinguish between failures and disturbances;
- implementing a fault management procedure;
- implementing a recovery strategy for each type of detected emergency;
- transmission of the engineering parameters upon request of the diagnostic instrument;
- controlling the system actuators (active diagnosis) through the specific controls;
- managing the emergency warning lamp(s), if foreseen;
- reprogramming the electronic system;
- storing statistic parameters.



Recognition of Failures emergencies



Recognition of failures – General rules

- The error information must be stored in the non-volatile memory;
- The control units that, for legislative and/or active safety reasons, must be able to update the non-volatile memory content with the data present in RAM also in case of a sudden or accidental power supply loss;
- System component diagnostics must be operative in all vehicle conditions – KeyOn, Cranking, Engine Running, Vehicle Running, Power Latch (or After Run);
- The electronic system must be able to recognize the cranking phase in order to filter any “disturbances” that could lead to the validation of false diagnoses;



Recognition of failures – Sensor/Actuator failures

- Electrical anomalies (O.C., S.C.G., S.C.B) on the signal and power supply lines of each sensor;
- Electrical anomalies (O.C., S.C.G., S.C.B) on the control and power supply lines of each actuator;
- Signal PLAUSIBILITY: software controls that are used to detect if a signal that originates from an individual sensor or from the feedback of an actuator is reliable or not;
- S.C.between different sensor/actuator signal lines; S.C. between signal lines and possible reference voltages;
- if the sensor/actuator power supply is the same for multiple components, a strategy must be foreseen (to be agreed upon with EE - EAI) that verifies the contemporaneity of a power supply failure on all sensors/actuators supplied by the same line.



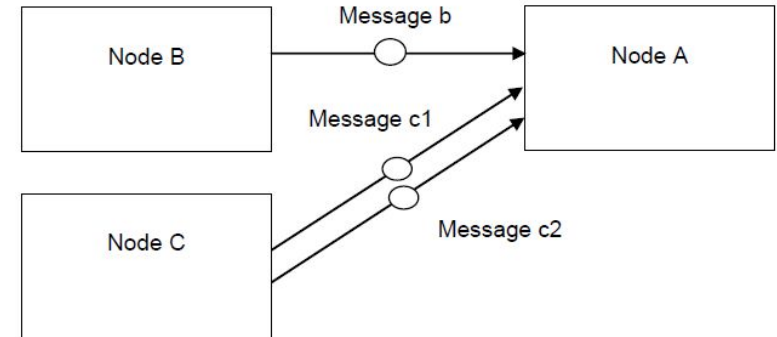
Recognition of failures – Other failures

- **Electronic system failures:** the electronic control system must also have self-diagnosis strategies for the electronic system components within the control unit that take part in permitting regular operations.
- **Power supply failures:** the electronic system must be able to diagnose any missing power supply connections, save the related error code in its memory and turn on the warning lamp if foreseen and available. If correct operation is guaranteed by a certain voltage range, the system that is supplied out of range must inhibit those diagnoses as they could generate false emergency recognition.
- **Mechanical part failures:** compatibly with hardware and software feasibility, it must be possible to keep under control, measure and control variations in mechanical parameters through the system sensor assembly or lines dedicated to monitoring.



Possible CAN failures – Node ABSENT

- Node ABSENT – Node A recognizes node C as absent from the network:
 - node A controls the reception of only ONE message from node C;
 - node A controls the presence of ALL messages transmitted by node C;
- there must be only one DTC assigned to each source node;
- the detection of this error must be disabled during engine cranking;
- the detection time (t_r) for this diagnosis must always be greater than the maximum value between the mute period for the **bus-off** specified for the source node ($t_{BUS-OFF}$) and the network input time of the source node (t_{Pon}) to avoid storing multiple DTC codes;
- this diagnosis must be inhibited immediately In case in which the receiving Node detected a **Node Mute** error on itself;
- maturation time: for BH and C CAN the minimum validation time is 1s and the maximum maturation time is 5s.





Recognition of failures – C.A.N. failures

The Nodes connected to the CAN network that diagnose and store diagnostic information must provide the following 2 bit coded information via the CAN network:

- **GenericFailSts** (Stored DTC);
- **CurrentFailSts** (Active DTC);

Usually, these information are present in the message that each ECU sends on CAN BUS with the description of the identifier as STATUS_C_XXX



Possible CAN failures – Node FAULTY

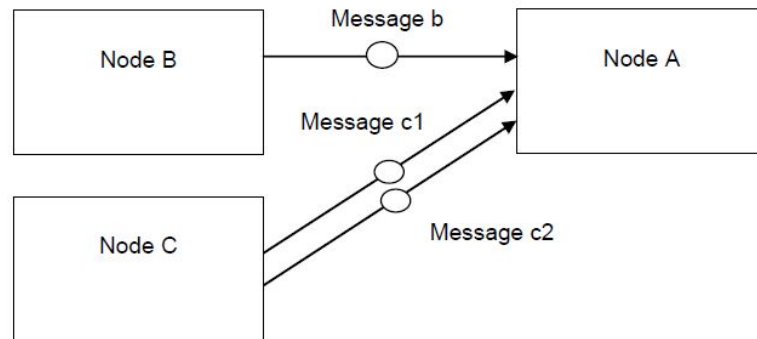
The implementation of this diagnosis is:

- **MANDATORY**: when one of the messages received has a shorter length;
- **OPTIONAL** if a received message is missing (in case of reception of more than one message) or receives a message in an incorrect format (e.g. wrong CRC or MC).

There must be only one DTC assigned to each source node;

The detection of this error must be disabled during the mute period that follows a bus-off condition.

The diagnosis of this error must be disabled as soon as the receiving Node detects a **Node Mute** error (on itself), and must be re-enabled when the **Node Mute** error becomes “not present”.

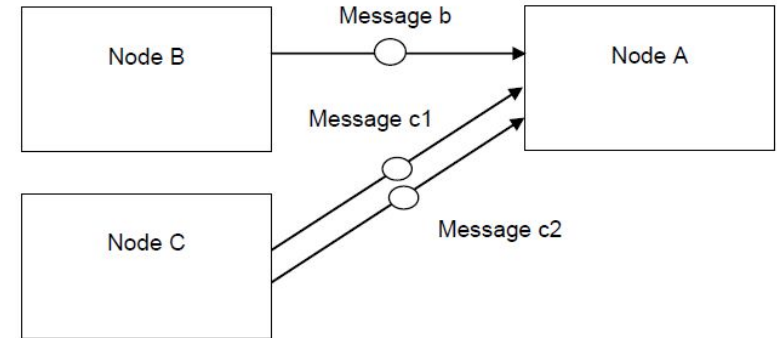




Possible CAN failures – Node MUTE

Node A recognizes that it is disconnected from the network. Node A can control its CAN Controller error status (error passive, warning), and the type of CAN Controller error (ACK error). In case of node MUTE, only the DTC related to the single node must be stored and the diagnosis of node absent and node faulty must be inhibited as long as the node mute condition persists because the diagnosis on other nodes carried out from a node with errors on its own controller should not be considered significant.

- the detection of this fault must disable the detection of the **Node ABSENT** and **Node FAULTY**;
- the detection time for this diagnosis must always be greater than the maximum value of the silence period for the bus-off specified for the source node;
- the detection time for this fault must be shorter than those adopted for Node **ABSENT** and Node **FAULTY** faults.



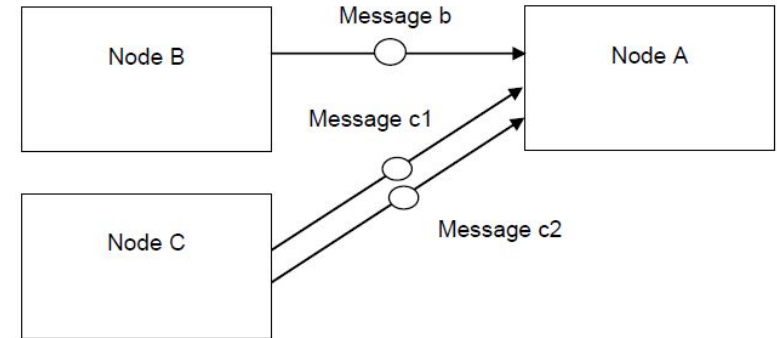


Possible CAN failures – BUS-Off

Node A detects a bus-off condition. Node A controls its CAN Controller error status (bus-off). In case of bus OFF, only the DTC related to the single node must be stored.

- During the mute period that must follow the detection of a bus-off, all faults connected to CAN communication must be disabled
- The Bus OFF event counter must be increased by one unit each time that the CAN controller enters the Bus OFF status in the same cycle.

BUS off Performance maturation timing is stated in: minimum time is 800ms and maximum time is 1500ms for each type of CAN BUS





Fault management



Error memory – Structure

The error memory is composed of cells that contain the following fields:

- **DTC Code (DTCMaskRecord):** is composed of 3 bytes; the first 2 bytes identify the component and the third byte (DTCLowByte statusOfDTC) identifies the type of fault (symptom);
- **StatusOfDTC:** identifies the status of the DTC at the moment of the reading with the diagnostic instrument;
- **DTCStatusMask:** each bit has the same meaning of the StatusOfDTC but it's used to require information;
- **DTCSnapshotRecordNumber:** identifies, in the error memory reading service, the record snapshot among those available in the ECU;
- **DTCExtendedDataRecordNumber:** identifies, in the error memory reading service, the record snapshot among those available in the ECU;
- **DTCSeverityMask:** each Supplier agrees with the EE - EAI systems engineer upon the rules for assigning the severity related to each DTC;
- **DTCSnapshotRecord:** this is made up of environmental parameters recorded at the moment of validating the fault and symptom that generated the storage.



Error memory – StatusOfDTC/DTCStatusMask

This byte takes on a specific meaning depending on if it is integrated in the tester request (Status Mask or in the ECU response (Status Of DTC). If inserted in the diagnostic instrument request to the ECU, it indicates on which Bits the comparison should be carried out with regard to the status values of each DTC, whereas in the central unit response, it unequivocally identifies the error status of each DTC. The definition of each status Mask or Status Of DTC Bit coincides as they have the same meaning independently of their context; the description follows:

- **bit 0:** testFailed
- **bit 1:** testFailedThisMonitoringCycle
- **bit 2:** pendingDTC
- **bit 3:** confirmedDTC
- **bit 4:** testNotCompletedSinceLastClear
- **bit 5:** testFailedSinceLastClear
- **bit 6:** testNotCompletedThisMonitoringCycle
- **bit 7:** warningIndicatorRequested



Error memory – StatusOfDTC/DTCStatusMask

bit 0: testFailed

In the absence of a fault in a present status or after a ClearDiagnosticInformation the Bit status, before executing a test, must be = 0

The bit must always indicate the result of the last test completed during the Monitoring Cycle:

- 1 = the result of the last completed test is Failed (fault present)
- 0 = the result of the last completed test is Passed (fault not present)

bit 1: testFailedThisMonitoringCycle

The default status, before the recognition of an anomaly or after a ClearDiagnosticInformation request, must be = 0.

The bit must indicate the result of the last test completed during the current Monitoring Cycle:

- 1 = during the current Monitoring Cycle at least one (1) test was completed whose result was Failed (fault detected).
- 0 = during the current Monitoring Cycle one (1) test has not yet been completed or all the completed tests have had a Passed result (no fault detected).



Error memory – StatusOfDTC/DTCStatusMask

bit 2: pendingDTC

The default status, before the recognition of an emergency or after a ClearDiagnosticInformation request, must be = 0.

The bit status must be set to 1 as soon as a test with a Failed result has been completed (emergency recognition).

The bit indicates the presence of an anomaly in the current Monitoring Cycle or during the previous Monitoring Cycle or the absence of an anomaly in the current and previous cycles:

- 0 = at the end of the current and previous Monitoring Cycles all completed tests have had a Passed result (a fault was **never** detected).
- 1 = during the current or previous Monitoring Cycle at least one (1) test was completed whose result was Failed (fault detected).

bit 3: confirmedDTC

The default status, before the recognition of an emergency, must be = 0.

The bit status must be set to 1 as soon as a test with a Failed result has been completed (emergency recognition).

The bit must indicate if the fault is present in the error memory (in the not volatile memory or in RAM while waiting to be stored in the not volatile memory):

- 1 = the DTC was confirmed at least once after the last clearDiagnosticInformation request and the criteria of automatic deletion from the not volatile memory were not satisfied.
- 0 = the DTC was NOT confirmed even once after the last clearDiagnosticInformation request or the criteria of automatic deletion from the not volatile memory were satisfied (the DTC was deleted from the not volatile memory).



Error memory – StatusOfDTC/DTCStatusMask

bit 6: testNotCompletedThisMonitoringCycle (optional)

The default status, at the start of each Monitoring Cycle, must be = 1.

The bit must indicate if the test was ever executed and completed during the current Monitoring Cycle (or executed and completed after the last clearDiagnosticInformation request):

- 1 = during the current Monitoring Cycle, the test of the diagnosis line under examination was never completed.
- 0 = during the current Monitoring Cycle the test was completed at least once independently of its result or the test was not yet activated due to unsatisfied enabling conditions.

bit 7: warningIndicatorRequested

The default status must be = 0.

The bit status must be set to 1 when the validated DTC activates the warning lamp.

The bit must indicate if the validated DTC is activating the switching on of the related lamp/message:

- 1 = the warning lamp is active; the DTC foresees the switching on of the warning lamp.
- 0 = the warning lamp is NOT active, the DTC does NOT foresee the switching on of the warning lamp or the enabling conditions were not satisfied (example error status or number of cycles for activation NOT reached).



Error memory – Snapshot record

The DTCSnapshotRecord is composed of the environmental parameter identifiers with the corresponding values frozen upon validation of the fault at the first and last occurrence.

The first 4 parameters of each DTCSnapshotRecord are obligatory for each system and DTC.

At the first validation of a fault, the first associated DTCSnapshotRecord must contain the data recorded at the moment of its first validation that remain until the DTC is deleted from the error memory.

The subsequent validations of the same fault (understood as devalidation and revalidation of the same fault - transition of Bit 0), will enable the composition of the second and last associated DTCSnapshotRecord that must contain the data recorded at its last validation



Error memory – Extended data record

The DTCExtendedDataRecord contains the following fields (see definition in the document):

- EventCounter (obligatory for all systems):
- Cycle to switch warning lamp OFF (obligatory for systems that manage a warning lamp)

Any other specific fields must be agreed upon between the Supplier and the EE - EAI systems engineer for each individual application and specified in the relative CDD document produced with CANdela.



Error memory – Event counter (no powertrain nodes)

The eventCounter parameter must be set to **40 decimals** in the following cases:

- at the moment in which the error cell is stored for the first time (coinciding with the simultaneous transition from 0 to 1 of confirmedDTC (bit 3 of status Of DTC) and of testFailed (bit 0 of StatusOfDTC).
- each time there is a transition from 0 to 1 of testFailedThisMonitoringCycle (Bit 1 of statusOfDTC) having confirmedDTC (bit 3 of status Of DTC) already at a value of 1 (error already stored previously in the error memory).
- The counter must be decreased by one unit at each Key ON if during the previous monitoring cycle (including the shut-down phase) the related error assumed the “*NOT Present*” status for the entire cycle (testFailed - bit 0 of StatusOfDTC set to 0).
- The error cell must be deleted from the error memory when the eventCounter reaches the value of 0.



Error memory – Event counter (Powertrain & EOBD)

The PowerTrain area electronic systems that have an impact on vehicle emissions must manage the “eventCounter” information according to the current European directive. In particular, the eventCounter must follow the following rules:

- the event counter must be set to **40 decimals** at the moment in which the error cell is stored for the first time (coinciding with the simultaneous transition from 0 to 1 of confirmedDTC (bit 3 of status Of DTC) and of testFailed (bit 0 of StatusOfDTC) and each time that there is a transition from 0 to 1 of testFailedThisMonitoringCycle (Bit 1 of statusOfDTC) having confirmedDTC (bit 3 of status Of DTC) already at a value of 1 (error already stored previously in the error memory).
- The counter will be decreased, within the monitoring cycle, if the MIL status is OFF at the beginning of the driving cycle, the test was completed with the result Passed and a **warm UP** is completed
- The error cell must be deleted from the error memory when the eventCounter reaches the value of 0 (cell not visible to the diagnostic instrument).



Error memory – Cycle to switch lamp off

The Electronic Systems that manage at least one warning lamp and/or message must manage the *Cycle to switch warning lamp OFF* parameter.

This parameter indicates the number of cycles (*Monitoring cycle*) that remain until the deactivation of the warning lamp, the following rules are valid (see example in fig. 3 and 4):

- if the fault does not foresee the lamp turning on, the *parameter* must always assume the value of 0;
- if the fault does foresee the lamp turning on, the parameter:
 - must be set to the number of cycles foreseen for turning off the warning lamp at the moment in which it is activated (for powertrain systems it is 3 for applications with EOBD diagnosis and 1 for systems without EOBD diagnosis)
 - must be decreased by one (1) unit at each consecutive error free monitoring cycle;
 - must be set to 0 in the moment in which the warning lamp is deactivated (warning lamp OFF);
 - must indicate the number of cycles necessary for turning off the warning light if the system is not in one of the above described conditions.



Error memory – Fault validation/de-validation filtering

- Each diagnosis line (DTC) must be specified in terms of timing/counter and enabling conditions for the necessary filtering in order to avoid not significant detections due to typical vehicle disturbances.
- It is a good practice to provide a series of time and counter variables for each diagnostic line in order to easily calibrate both validation as well as devalidation without having to make changes to the control unit sw.
- Based on the type of ECU hardware, the Supplier must make the detailed information about the anomaly available, properly distinguishing the symptom of each DTC from among the electric type diagnoses and the plausibility diagnoses, in addition, each DTC must identify a specific component (sensor / actuator) with exception of the plausibility diagnoses where the original cause of the anomaly cannot be identified with certainty, in this case the Supplier will agree with the system engineer upon the selection of the DTC and the specific symptom.





Error memory – Erasing

- It must be possible to delete, through a specific diagnostic command, the entire contents of the error memory (both *volatile* and *non-volatile* memory);
- The cancellation of the *volatile* memory must occur upon reception of the diagnostic instrument command, all of the memory cells related to faults having Bit 0 TestFailed of status Of DTC must be reset to 0 “Fault Not Present”;
- The error cells in which the eventCounter decreases down to 0 must be deleted autonomously by the electronic system. Other types of automatic cancellation are not permitted in addition to those cited



Error memory – Recovery strategies

At the verification of each anomaly, the electronic system must be able to implement the required recovery strategy. These strategies have the purpose of:

- avoiding further damage to the system;
- permitting system operation, even if with degraded performances;
- protecting the safety of the driver and the system in general.

In case of sensor malfunctions, the electronic system will replace the values obtained from the faulty sensors using the recovery values agreed upon with the systems engineer or, if possible, will calculate them obtaining them indirectly by reading/processing other parameters.

In case of actuator emergencies, the recovery strategies to adopt depend on the actuator in question.

All the recovery strategies, both those related to sensors as well as those related to actuators, are differentiated by component type and must therefore be agreed upon with the systems engineer on a case by case basis and specified in detail in the Cdd CANdela and diagnostic function summary.



Error memory – Warning lamp management (PWT & EOBD)

The MIL warning lamp must remain on from Key ON until the engine is started and must be turned off after the engine is ignited if no faults are detected. The operation of the MIL warning lamp is described in the current European directive.

The turning on of the MIL lamp may be of the type On-1, On-3 or Off depending if the outfitting was type-approved according to EOBD regulations or with the exclusion of the EOBD diagnostic strategies. The MIL activation modality is described briefly below:

- **ON-1** : the MIL is activated immediately upon error validation;
- **ON-3** : the MIL is activated at the third cycle if the error was validated for three consecutive monitoring cycles;
- **OFF** : the activation of the MIL is not foreseen in any case for this type of error as it is not considered serious for safety purposes and is not emission relevant.



Read data by Identifier



RDI – Engineering parameter transmission

All engineering parameters generated by the sensors and processed by the electronic system as well as some parameters calculated and used for control and implementation must be available on the diagnostic link.

These parameters will be made available on the diagnostic link, upon proper request, individually or in a group (with a specific command); in addition, they must be accessible in the key ON status, with the motor running and at any vehicle speed.

In case of errors on a certain component, the related parameter that is transmitted must be the one actually read by the sensor and not the recovery parameter used by the electronic system.

If one or more parameters are not present in an application (version, outfitting), upon request by the instrument of that specific parameter, the control unit must respond with a negative response whose significance must be “parameter not available” (Nack 0x31).



RDI – Mandatory parameters

- RDI \$1008 ECU Time Stamps (RAM)
- RDI \$1009 ECU Time Stamps from Key ON (RAM)
- RDI \$F186 (active diagnostic session data identifier)
- RDI \$2001 (Odometer in EEPROM)
- RDI \$2003 (Number of Flash ROM re-writing)
- RDI \$2008 (ECU Time Stamps stored in EEPROM)
- RDI \$2009 (ECU Time Stamps from Key On, stored in EEPROM)
- RDI \$200A (Key On counter)
- RDI \$6080 (Event Counter)
- RDI \$6081 (Cycle to switch warning lamp OFF)
- RDI \$6082 (DTC failure type byte)



RDI – Identifier

- RDI \$F187 (Vehicle manufacturer spare part number)
- RDI \$F18C (ECU serial number)
- RDI \$F190 (V.I.N)
- RDI \$F192 (System supplier ECU Hw number)
- RDI \$F193 (System supplier ECU Hw version number)
- RDI \$F194 (System supplier ECU Sw number)
- RDI \$F195 (System supplier ECU Sw version number)
- RDI \$F196 (Exhaust regulation or type approval number)
- RDI \$F1A0 (Identification code)
- RDI \$F1A5 (ISO code)



I-O Control



I/O Control – Active diagnosis

The active diagnosis is a function that, through specific diagnostic controls, permits to control the system actuators or modify the value of magnitudes of system output that is transmitted to other electronic systems.

During active diagnosis, the electronic system must implement the diagnostic strategies present during normal system operations.

Active diagnosis must be foreseen in the following situations:

- the component is not continuously monitored by the ECU or must be diagnosed only in particular conditions that are difficult to reach; through active diagnosis it is instead possible to control the component to immediately verify its functionality by reading the error memory and through specific engineering parameters
- component activation must be verified by sound or sight
- the component must be calibrated

whereas it must be avoided:

- on signals present on CAN networks that are used for plausibility controls from other electronic systems; in fact plausibility diagnostic controls should not enter into action when the surrounding situation is not correct.



Statistical function



Statistical – function

The diagnostic functions requested of the system may include statistical functions that permit to reconstruct the history of the vehicle, the electronic system or information that can be used to reconstruct the use of the system itself. Information can also be stored regarding the total number of missions, the number of missions in which anomalies have been verified, the number of actuations of a particular component and anything considered necessary by departments that use the functions. The main statistical function are:

- Engine Overspeed” Function
- “Odometer” Function



Routine



Routine – Calibration, settings, learning

Normally, each control unit / component must guarantee correct functionality as soon as it is installed on the vehicle and powered. However, there are particular cases for which, in order to guarantee perfect functionality, procedures must be carried out using the diagnostic instruments. For **example**, the following list is provided:

- programming keys and/or remote controls;
- parameter characteristic learning (throttle body, phonic wheel, gear position, mixture title...);
- sensor and/or actuator calibration (steering angle, radar beams...);
- operations resulting from or necessary for disconnecting/connecting control units/components (odometer, service intervals, transmission oil depressurization...

If not done properly, all of these procedures can jeopardize correct system operation and as a result must be defined in the document produced with CANdela in great detail. Therefore, the Supplier must indicate for each procedure:

- why it must be executed.
- when it must be executed.
- which operation must be performed by repair shop personnel (if foreseen).
- which conditions / sequences must be respected to execute the procedure.
- which commands the diagnostic instrument must send.
- which results does the control unit send to the instrument for each command received.
- which time periods must be respected.
- The conditions in which the procedure appears to be correctly completed.



Diagnostic services



RDBI – Service \$22

The ReadDataByIdentifier service allows the tester to request data record values from the ECU identified by one or more recordDataIdentifiers

A_Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	ReadDataByIdentifier Request Service Id	M	22	RDBI
#2	recordDataIdentifier[] #1 = [byte 1 (MSB) byte 2 (see par 5.4.1.2.2)]	M	00-FF	RDI_ B1
#3		M	00-FF	B2
:	:	:	:	:
#n-1	recordDataIdentifier[] #m = [byte 1 (MSB) byte 2 (see par 5.4.1.2.2)]	U	00-FF	RDI_ B1
#n		U	00-FF	B2

A_Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	ReadDataByIdentifier Response Service Id	M	62	RDBIPR
#2	recordDataIdentifier[] #1 = [byte 1 (MSB) byte 2]	M	00-FF	RDI_ B1
#3		M	00-FF	B2
#4	dataRecord[] #1 = [data#1 : data#m]	M	00-FF	DREC_ DATA_1
:		:	:	:
#(m-1)+4		U	00-FF	DATA_m



I/O Control – Service \$2F

The service allows the control of a single dataIdentifier with the corresponding controlOptionRecord in a single request message. Doing so the server will respond with a single response message including the dataIdentifier of the request message plus optional controlStatus information.

A_Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	InputOutputControlByIdentifier Request Service Id	M	2F	IOCBI
#2	dataIdentifier#1[] = [byte 1 (MSB) byte 2 (LSB) (see par 5.6.1.2.2)]	M	00-FF	IOI_
#3		M	00-FF	B1 B2
#4	controlOptionRecord#1[] = [controlState#1/inputOutputControlParameter : controlState#m(see par 5.6.1.2.2)]	M ₁	00-FF	CSR_
#4+(m-1)		C ₁	00-FF	IOCP_/CS_ : CS_
#4+m	controlEnableMaskRecord#1[] = [controlMask#1 : controlMask#r(see par 5.6.1.2.2)]	C ₂	00-FF	CEM_
#4+m+(r-1)		C ₂	00-FF	CM_ : CM_

A_Data byte	Parameter Name	Cvt	Hex Value	Mnemonic	
#1	InputOutputControlByIdentifier Response Service Id	S	6F	IOCBIPR	
#2	dataIdentifier#1[] = [byte 1 (MSB) byte 2 (LSB)]	M	00-FF	IOI_	
#3		M	00-FF	B1 B2	
#4	controlStatusRecord#1[] = [controlState#1/inputOutputControlParameter : controlState#m]	C ₁	00-FF	CSR_	
#4+(m-1)		:	:	:	
		C ₂	00-FF	IOCP_/CS	
		:	:	CS_	
C ₁ :	The presence of this parameter depends on its usage in the request message. ControlState#1 is either used as an InputOutputControlParameter or as an additional controlState. If it is used as an InputOutputControlParameter then it shall be present in the response message and shall be the echo of the InputOutputControlParameter value given in the request message. In all other cases it is user optional to be present (depends on the usage of a controlStatusRecord).				
C ₂ :	The presence of this parameter depends on the inputOutputIdentifier and the inputOutputControlParameter (if controlState#1 is used as an inputOutputControlParameter).				

Hex	Description	Cvt	Mnemonic
00	returnControlToECU This value shall indicate to the ECU that the tester does no longer have control about the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier. Number of controlState bytes in request : 0 Number of controlState bytes in pos. response: depends on the inputOutputIdentifier	M	RCTECU
01	resetToDefault This value shall indicate to the ECU that it is requested to reset the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier to its default state. Number of controlState bytes in request : 0 Number of controlState bytes in pos. response: depends on the inputOutputIdentifier	M	RTD
02	freezeCurrentState This value shall indicate to the ECU that it is requested to freeze the current state of the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier. Number of controlState bytes in request : 0 Number of controlState bytes in pos. response: depends on the inputOutputIdentifier	U	FCS
03	shortTermAdjustment This value shall indicate to the ECU that it is requested to adjust the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier in RAM to the value(s) included in the controlOption parameter(s). (e.g. set Idle Air Control Valve to a specific step number, set pulse width of valve to a specific value/duty cycle). Number of controlState bytes in request : depends on the inputOutputIdentifier Number of controlState bytes in pos. response: depends on the inputOutputIdentifier	M	STA
04 - FF	reservedByDocument This value is reserved by this document for future definition.	M	RBD



Routine – Service \$31

The RoutineControl service is used by the tester to:

- start a routine,
- stop a routine, and
- request routine results
- referenced by a 2-byte routineIdentifier.

A_Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	RoutineControl Response Service Id	S	71	RCPR
#2	routineControlType	M	00-FF	RCTP_
#3	routineIdentifier [] = [byte 1 (MSB)	M	00-FF	RI_ B1
#4	byte 2]	M	00-FF	B2
#5	routineStatusRecord[] = [routineStatus#1	U	00-FF	RSR_ RS_
:	:	:	:	:
#n	routineStatus#m]	U	00-FF	RS_

A_Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	RoutineControl Request Service Id	M	31	RC
#2	sub-function = [routineControlType (see Table 92)]	M	00-FF	LEV_ RCTP_
#3	routineIdentifier [] = [byte 1 (MSB)	M	00-FF	RI_ B1
#4	byte 2 Table 93]	M	00-FF	B2
#5	routineControlOptionRecord[] = [routineControlOption#1	C/U	00-FF	RCEOR_ RCO_
:	:	:	:	:
#n	routineControlOption#m]	C/U	00-FF	RCO_
C: This parameter is user optional to be present for sub-function parameter startRoutine and stopRoutine.				

Hex (bit 6-0)	Description	Cvt	Mnemonic
00	reservedByDocument This value is reserved by this document for future definition.	M	RBD
01	startRoutine This parameter specifies that the ECU shall start the routine specified by the routineIdentifier.	M	STR
02	stopRoutine This parameter specifies that the ECU shall stop the routine specified by the routineIdentifier.	U	STPR
03	requestRoutineResults This parameter specifies that the ECU shall return result values of the routine specified by the routineIdentifier.	M	RRR
04 - 7F	reservedByDocument This value is reserved by this document for future definition.	M	RBD



DTC – By Status Mask Service \$19 \$02

This sub function of the service \$19 (Read DTC Information) is useful to obtain all the DTC stored in ECU that has a specific StatusOfDTC

A_Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	ReadDTCInformation request Service Id	M	19	RDI
#2	sub-function = [reportDTCByStatusMask (see Table 74)]	M	02	LEV_ RDTCSBM
#3	DTCStatusMask (see Table 75)	M	00-FF	DTCSM

A_Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	ReadDTCInformation response Service Id	M	59	RDIPR
#2	reportType = [reportDTCByStatusMask]	M	02	LEV_ RDTCSBM
#3	DTCStatusAvailabilityMask	M	00-FF	DTCSAM
	DTCAndStatusRecord[] = [			DTCSAR_
#4	DTCHighByte#1	C ₁	00-FF	DTCHB
#5	DTCMiddleByte#1	C ₁	00-FF	DTCMB
#6	DTCLowByte#1	C ₁	00-FF	DTCLB
#7	statusOfDTC#1	C ₁	00-FF	SODTC
#8	DTCHighByte#2	C ₂	00-FF	DTCHB
#9	DTCMiddleByte#2	C ₂	00-FF	DTCLB
#10	DTCLowByte#2	C ₂	00-FF	DTCLB
#11	statusOfDTC#2	C ₂	00-FF	SODTC
:	:	:	:	:
#n-3	DTCHighByte#m	C ₂	00-FF	DTCHB
#n-2	DTCMiddleByte#m	C ₂	00-FF	DTCMB
#n-1	DTCLowByte#m	C ₂	00-FF	DTCLB
#n	statusOfDTC#m]	C ₂	00-FF	SODTC



DTC – SnapshotRecord Service \$19 \$04

This sub function of the service \$19 (Read DTC Information) is useful to obtain the Snapshot Record of a specific DTC that is stored in ECU. Each ECU, for each DTC can store two different Snapshot (the first Snapshot has the information about the first DTC validation and the second Snapshot has the information about the last DTC validation).

A_Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	ReadDTCInformation request Service Id	M	19	RDI
#2	sub-function = [reportDTCSnapshotRecordByDTCNumber (see Table 74)]	M	04	LEV_ RDTCSBBDT C
#3	DTCMaskRecord[] = [DTCHighByte	M	00-FF	DTCMREC_ DTCHB
#4	DTCMiddleByte	M	00-FF	DTCMB
#5	DTCLowByte (see Table 75)]	M	00-FF	DTCLB
#6	DTCSnapshotRecordNumber (see Table 75)	M	00-FF	DTCSSRN

A_Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	ReadDTCInformation response Service Id	M	59	RDIPR
#2	reportType = [reportDTCSnapshotRecordByDTCNumber]	M	04	LEV_ RDTCSBBDT C
#3	DTCAndStatusRecord[] = [DTCHighByte	M	00-FF	DTCASR_ DTCHB
#4	DTCMiddleByte	M	00-FF	DTCMB
#5	DTCLowByte	M	00-FF	DTCLB
#6	statusOfDTC]	M	00-FF	SODTC
#7	DTCSnapshotRecordNumber #1	C ₁	00-FF	DTCSSRN
#8	DTCSnapshotRecordNumberOfIdentifiers #1	C ₁	00-FF	DTCSSRNI
#9	DTCSnapshotRecord[] #1 = [recordDataIdentifier#1 byte #1 (MSB)	C ₁	00-FF	DTCSSR_ RDIb11
#10	recordDataIdentifier#1 byte #2	C ₁	00-FF	RDIb12
#11	snapshotData#1 byte #1	C ₁	00-FF	SSD11
:	:	C ₁	:	:
#11+(p-1)	snapshotData#1 byte #p	C ₁	00-FF	SSD1p
:	:	C ₁	:	:
#r-(m-1)-2	recordDataIdentifier#w byte #1 (MSB)	C ₂	00-FF	RDIb21
#r-(m-1)-1	recordDataIdentifier#w byte #2	C ₂	00-FF	RDIb22
#r-(m-1)	snapshotData#w byte #1	C ₂	00-FF	SSD21
:	:	C ₂	:	:
#r	snapshotData#w byte #m]	C ₂	00-FF	SSD2m
:	:	:	:	:
#1	DTCSnapshotRecordNumber #x	C ₃	00-FF	DTCSSRN
#1+1	DTCSnapshotRecordNumberOfIdentifiers #x	C ₃	00-FF	DTCSSRNI
#1+2	DTCSnapshotRecord[] #x = [recordDataIdentifier#1 byte #1 (MSB)	C ₃	00-FF	DTCSSR_ RDIb11
#1+3	recordDataIdentifier#1 byte #2	C ₃	00-FF	RDIb12
#1+4	snapshotData#1 byte #1	C ₃	00-FF	SSD11
:	:	C ₃	:	:
#1+4+(p-1)	snapshotData#1 byte #p	C ₃	00-FF	SSD1p
:	:	C ₃	:	:
#n-(u-1)-2	recordDataIdentifier#w byte #1 (MSB)	C ₄	00-FF	RDIb21
#n-(u-1)-1	recordDataIdentifier#w byte #2	C ₄	00-FF	RDIb22
#n-(u-1)	snapshotData#w byte #1	C ₄	00-FF	SSD21
:	:	C ₄	:	:
#n	snapshotData#w byte #u]	C ₄	00-FF	SSD2u



DTC – ExtendedDataRecord Service \$19 \$06

This sub function of the service \$19 (Read DTC Information) is useful to obtain the Extended Data Record of a specific DTC that is stored in ECU. Usually the EDR has two information inside: Event Counter and Cycle to switch Lamp Off).

A_Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	ReadDTCInformation request Service Id	M	19	RDI
#2	sub-function = [reportDTCExtendedDataRecordByDTCNumber] (see Table 74)	M	06	LEV_ RDTCEDRBD
#3	DTCMaskRecord[] = [DTCHighByte	M	00-FF	DTCMREC_ DTCHB
#4	DTCMiddleByte	M	00-FF	DTCMB
#5	DTCLowByte(see Table 75)]	M	00-FF	DTCLB
#6	DTCExtendedDataRecordNumber (see Table 75)	M	00-FF	DTCEDRN

A_Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	ReadDTCInformation response Service Id	M	59	RDI PR
#2	reportType = [reportDTCExtendedDataRecordByDTCNumber]	M	06	LEV_ RDTCEDRBD
#3	DTCAndStatusRecord[] = [DTCHighByte	M	00-FF	DTCASR_ DTCHB
#4	DTCMiddleByte	M	00-FF	DTCMB
#5	DTCLowByte	M	00-FF	DTCLB
#6	statusOfDTC]	M	00-FF	SODTC
#7	DTCExtendedDataRecordNumber #1	C ₁	00-FF	DTCEDRN
#8	DTCExtendedDataRecord [] #1 = [extendedData #1 byte #1	C ₁	00-FF	DTCSSR_ EDD11
:	:	C ₁	:	:
#8+(p-1)	extendedData #1 byte #p]	C ₁	00-FF	EDD1p
:	:	:	:	:
#t	DTCExtendedDataRecordNumber #x	C ₂	00-FF	DTCEDRN
#t+1	DTCExtendedDataRecord [] #x = [extendedData #x byte #1	C ₂	00-FF	DTCSSR_ EDDx1
:	:	C ₂	00-FF	:
#t+1+(p-1)	extendedData #x byte #p]	C ₂	00-FF	EDDxp

C₁: The DTCExtendedDataRecordNumber and the extendedData in the DTCExtendedDataRecord parameter are only present if at least one DTCExtendedDataRecord is available to be reported (DTCExtendedDataRecordNumber unequal to FF hex in the request or only one record is available to be reported if DTCExtendedDataRecordNumber is set to FF hex in the request).

C₂: The DTCExtendedDataRecordNumber and the extendedData in the DTCExtendedDataRecord parameter are only present if all records are requested to be reported (DTCExtendedDataRecordNumber set to FF hex in the request) and more than one record is available to be reported.



DTC – ReportNumberOfDTCBySeverityMaskRecord Service \$19 \$07

This sub function of the service \$19 (Read DTC Information) is useful to obtain the number of the DTC stored that has a specific severity and that has a specific StatusOfDTC

A_Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	ReadDTCInformation request Service Id	M	19	RDI
#2	sub-function = [reportNumberOfDTCBySeverityMaskRecord reportDTCBySeverityMaskRecord (see Table 74)]	M	07	LEV_ RNODTCBSMR
			08	RDTCSMR
#3	DTCSeverityMaskRecord[] = [DTCSeverityMask	M	00-FF	DTCSMREC_ DTCSM
#4	DTCStatusMask (see Table 75)]	M	00-FF	DTCSM

A_Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	ReadDTCInformation response Service Id	M	59	RDIPR
#2	reportType = [reportNumberOfDTCBySeverityMaskRecord]	M	07	LEV_ RNODTCBSMR
#3	DTCStatusAvailabilityMask	M	00-FF	DTCSAM
#4	DTCFormatIdentifier = [ISO15031-6DTCFormat ISO14229-1DTCFormat SAEJ1939-73DTCFormat ISO11992-4DTCFormat]	M	00 01 02 03	DTCFID_ 15031-6DTCF 14229-1DTCF J1939-73DTCF 11992-4DTCF
#5	DTCCount[] = [DTCCountHighByte	M	00-FF	DTCC_ DTCCHB
#6	DTCCountLowByte]	M	00-FF	DTCCLB



DTC – ReportDTCBySeverityMaskRecord Service \$19 \$08

This sub function of the service \$19 (Read DTC Information) is useful to obtain the list of the DTC stored that has a specific severity and that has a specific StatusOfDTC

A_Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	ReadDTCInformation request Service Id	M	19	RDI
#2	sub-function = [	M		LEV_
	reportNumberOfDTCBySeverityMaskRecord		07	RNODTCBSMR
	reportDTCBySeverityMaskRecord (see Table 74))		08	RDTCSMR
#3	DTCSeverityMaskRecord[] = [	M	00-FF	DTCSEVREC_
#4	DTCStatusMask (see Table 75))	M	00-FF	DTCSEVREC_

A_Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	ReadDTCInformation response Service Id	M	59	RDIPR
#2	reportType = [	M		LEV_
	reportDTCBySeverityMaskRecord		08	RDTCSMR
	reportSeverityInformationOfDTC]		09	RSIODTC
#3	DTCStatusAvailabilityMask	M	00-FF	DTCSEVREC_
	DTCAndSeverityRecord[] = [			DTCSEVREC_
#4	DTCSeverity #1	C ₁	00-FF	DTCSEVREC_
#5	DTCFunctionalUnit #1	C ₁	00-FF	DTCSEVREC_
#6	DTCHighByte #1	C ₁	00-FF	DTCSEVREC_
#7	DTCMiddleByte #1	C ₁	00-FF	DTCSEVREC_
#8	DTCLowByte #1	C ₁	00-FF	DTCSEVREC_
#9	statusOfDTC #1	C ₁	00-FF	SODTC
	:		:	:
	DTCSeverity #m	C ₂	00-FF	DTCSEVREC_
#n-5	DTCFunctionalUnit #m	C ₂	00-FF	DTCSEVREC_
#n-4	DTCHighByte #m	C ₂	00-FF	DTCSEVREC_
#n-3	DTCMiddleByte #m	C ₂	00-FF	DTCSEVREC_
#n-2	DTCLowByte #m	C ₂	00-FF	DTCSEVREC_
#n-1	statusOfDTC #m]	C ₂	00-FF	SODTC



DTC – ReportSeverityInfo Service \$19 \$09

This sub function of the service \$19 (Read DTC Information) is useful to obtain the severity of a specific DTC

A Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	ReadDTCInformation request Service Id	M	19	RDI
#2	sub-function = [reportSeverityInformationOfDTC] (see Table 74)	M	09	LEV_ RSIODTC
#3	DTCMaskRecord[] = [DTCHighByte DTCMiddleByte DTCLowByte (see Table 75)]	M	00-FF	DTCMREC_
#4		M	00-FF	DTCHB
#5		M	00-FF	DTCMB
#5		M	00-FF	DTCLB

A Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	ReadDTCInformation response Service Id	M	59	RDIPR
#2	reportType = [reportDTCBySeverityMaskRecord reportSeverityInformationOfDTC]	M	08 09	LEV_ RDTCSMR RSIODTC
#3	DTCStatusAvailabilityMask	M	00-FF	DTCSAM
#4	DTCAndSeverityRecord[] = [DTCSeverity #1 DTCFunctionalUnit #1 DTCHighByte #1 DTCMiddleByte #1 DTCLowByte #1 statusOfDTC #1 : DTCSeverity #m DTCFunctionalUnit #m DTCHighByte #m DTCMiddleByte #m DTCLowByte #m statusOfDTC #m]	C ₁	00-FF	DTCSA_
#5		C ₁	00-FF	DTCS
#6		C ₁	00-FF	DTCSF
#7		C ₁	00-FF	DTCHB
#8		C ₁	00-FF	DTCMB
#9		C ₁	00-FF	DTCLB
:		:	:	SODTC
:		:	:	:
#n-5		C ₂	00-FF	DTCS
#n-4		C ₂	00-FF	DTCSF
#n-3		C ₂	00-FF	DTCHB
#n-2		C ₂	00-FF	DTCMB
#n-1		C ₂	00-FF	DTCLB
#n		C ₂	00-FF	SODTC



DTC – FirstConfirmedDTC Service \$19 \$0C

This sub function of the service \$19 (Read DTC Information) is useful to obtain the First Confirmed DTC that is stored in error memory. The First Confirmed DTC is the oldest DTC stored.

A_Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	ReadDTCInformation request Service Id	M	19	RDI
#2	sub-function = [reportFirstConfirmedDTC reportMostRecentConfirmedDTC (see Table 74)]	M	0C 0E	LEV_ RFCDTC RMRVDTC

reportMostRecentConfirmedDTC				
A_Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	ReadDTCInformation response Service Id	M	59	RDIPR
#2	reportType = [reportFirstConfirmedDTC reportMostRecentConfirmedDTC]	M	0C 0E	LEV_ RFCDTC RMRVDTC
#3	DTCStatusAvailabilityMask	M	00-FF	DTCSAM
#4	DTCAndStatusRecord[] = [DTCHighByte#1	C ₁	00-FF	DTCHB ₁
#5	DTCMiddleByte#1	C ₁	00-FF	DTCMB ₁
#6	DTCLowByte#1	C ₁	00-FF	DTCLB ₁
#7	statusOfDTC#1	C ₁	00-FF	SODTC ₁
#8	DTCHighByte#2	C ₂	00-FF	DTCHB ₂
#9	DTCMiddleByte#2	C ₂	00-FF	DTCLB ₂
#10	DTCLowByte#2	C ₂	00-FF	DTCLB ₂
#11	statusOfDTC#2	C ₂	00-FF	SODTC ₂
:	:	:	:	:
#n-3	DTCHighByte#m	C ₂	00-FF	DTCHB _m
#n-2	DTCMiddleByte#m	C ₂	00-FF	DTCMB _m
#n-1	DTCLowByte#m	C ₂	00-FF	DTCLB _m
#n	statusOfDTC#m]	C ₂	00-FF	SODTC _m

C₁: This parameter is only present if reportType = reportSupportedDTCs, reportDTCByStatusMask



DTC – MostRecentDTC Service \$19 \$0E

This sub function of the service \$19 (Read DTC Information) is useful to obtain the Most Recent Confirmed DTC that is stored in error memory.

A_Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	ReadDTCInformation request Service Id	M	19	RDI
#2	sub-function = [reportFirstConfirmedDTC reportMostRecentConfirmedDTC (see Table 74)]	M	0C 0E	LEV_ RFCDTC RMRVDTC

reportMostRecentConfirmedDTC				
A_Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	ReadDTCInformation response Service Id	M	59	RDIPR
#2	reportType = [reportFirstConfirmedDTC reportMostRecentConfirmedDTC]	M	0C 0E	LEV_ RFCDTC RMRVDTC
#3	DTCStatusAvailabilityMask	M	00-FF	DTCSAM
	DTCAndStatusRecord[] = [DTCHighByte#1 DTCMiddleByte#1 DTCLowByte#1 statusOfDTC#1 DTCHighByte#2 DTCMiddleByte#2 DTCLowByte#2 statusOfDTC#2 : DTCHighByte#m DTCMiddleByte#m DTCLowByte#m statusOfDTC#m]	C ₁ C ₁ C ₁ C ₁ C ₂ C ₂ C ₂ C ₂ : C ₂ C ₂ C ₂ C ₂	00-FF 00-FF 00-FF 00-FF 00-FF 00-FF 00-FF 00-FF : 00-FF 00-FF 00-FF 00-FF	DTCHB_ DTCHB DTCHB DTCLB SODTC DTCHB DTCLB DTCLB SODTC : DTCHB DTCHB DTCLB SODTC

(C₁ : This parameter is only present if reportType = reportSupportedDTCs, reportDTCByStatusMask



Diagnostic services – Negative responses

ResponseCode parameter values are detailed in ISO 14229, that contains a list of suggested values for responseCode to be used for all specified services

ResponseCodes	ServiceIdentifiers																		
	(10h) DiagnosticSessionControl	(11h) ECUReset	(27h) securityAccess	(28 hex) CommunicationControl	(3E hex) TesterPresent	(83 hex) AccessTimingParameter	(84 hex) SecuredDataTransmission	(85 hex) ControlDTCSetting	(86 hex) ResponseOnEvent	(87 hex) LinkControl	(22 hex) ReadDataByIdentifier	(23 hex) ReadMemoryByAddress	(24 hex) ReadScalingDataByIdentifier	(2A hex) ReadDataByPeriodicIdentifier	(2C hex) DynamicallyDefinedDataIdentifier	(2E hex) WriteDataByIdentifier	(3D hex) WriteMemoryByAddress	(14 hex) ClearDiagnosticInformation	(19 hex) ReadDTCInformation
(\$10) generalReject																			
(\$11) serviceNotSupported	\$	\$	\$	\$	\$	\$	\$	\$	\$	\$	\$	\$	\$	\$	\$	\$	\$	\$	\$
(\$12) subFunctionNotSupported	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√
(\$13) incorrectMessageLengthOrInvalidFormat	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√
(\$21) busy-repeatRequest	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖
(\$22) conditionsNotCorrect	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√
(\$24) RequestSequenceError	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√
(\$31) requestOutOfRange			√	√			√	√	√	√	√	√	√	√	√	√	√	√	√
(\$33) securityAccessDenied			√	√						√				√	√	√	√	√	√
(\$35) invalidKey			√																
(\$36) exceedNumberOfAttempts			√																
(\$37) requiredTimeDelayNotExpired			√																
(\$70) uploadDownloadNotAccepted																			
(\$71) transferDataSuspended																			
(\$72) generalProgrammingFailure																			
(\$73) wrongBlockSequenceCounter																			

ResponseCodes	ServiceIdentifiers																		
	(10h) DiagnosticSessionControl	(11h) ECUReset	(27h) securityAccess	(28 hex) CommunicationControl	(3E hex) TesterPresent	(83 hex) AccessTimingParameter	(84 hex) SecuredDataTransmission	(85 hex) ControlDTCSetting	(86 hex) ResponseOnEvent	(87 hex) LinkControl	(22 hex) ReadDataByIdentifier	(23 hex) ReadMemoryByAddress	(24 hex) ReadScalingDataByIdentifier	(2A hex) ReadDataByPeriodicIdentifier	(2C hex) DynamicallyDefinedDataIdentifier	(2E hex) WriteDataByIdentifier	(3D hex) WriteMemoryByAddress	(14 hex) ClearDiagnosticInformation	(19 hex) ReadDTCInformation
(\$78) requestCorrectlyReceived-ResponsePending	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖	⊖
(\$7E) subFunctionNotSupportedInActiveSession	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%
(\$7F) serviceNotSupportedInActiveSession	% ¹⁾	% ¹⁾	% ¹⁾	% ¹⁾	% ¹⁾	% ¹⁾	% ¹⁾	% ¹⁾	% ¹⁾	% ¹⁾	% ¹⁾	% ¹⁾	% ¹⁾	% ¹⁾	% ¹⁾	% ¹⁾	% ¹⁾	% ¹⁾	% ¹⁾
(\$81) rpmTooHigh		√	√	√															
(\$82) rpmTooLow		√	√	√															
(\$83) engineIsRunning		√	√	√															
(\$84) engineIsNotRunning		√	√	√															
(\$85) engineRunTimeTooLow																			
(\$86) temperatureTooHigh																			
(\$87) temperatureTooLow																			
(\$88) vehicleSpeedTooHigh																			
(\$89) vehicleSpeedTooLow																			
(\$8A) throttlePedalTooHigh																			
(\$8B) throttlePedalTooLow																			
(\$8C) transmissionRangeInNeutral																			
(\$8D) transmissionRangeInGear																			
(\$8F) brakeSwitch(es)NotClosed (Brake Pedal not pressed or not applied)																			
(\$90) shifterLeverNotInPark																			
(\$91) torqueConverterClutchLocked																			
(\$92) VoltageTooHigh																			
(\$93) VoltageTooLow																			



PROXI



PROXI

Car Configuration

End Of Line programming is used to limit the number of different systems to be produced for satisfying the product grid requirements, when the differences among various configurations of a certain system can be managed by changing one or more parameters in the SW.

The parameters are changed through the transmission of the customizing data.

The data package forming the system configuration data is called PROXI file.

PROXI – Programming and configuration of integrated systems

The EOL programming can be executed only using diagnosis protocol and functions, so it will be possible to program only systems equipped with standard diagnosis on can network.



PROXI

Car Configuration

The master node of the proxy is the body computer (BCM).

A secondary master node, usually the instrument cluster (IPC), is used for backup functions.

The master node stores all vehicle configuration information and allows the diagnostic tool to read them.

The other nodes taking part to the proxy procedure store only the information directly concerning them, and only when the writing phase is properly completed, they store such informations in their permanent memory, activating their own functionalities according with it.



PROXI

EOL Programming file format

The set of all the informations transmitted to the nodes taking part in the PROXI procedure is called PROXI file.

The PROXI format can be .BYT, .EOL, .EPLUS, .HEX.

For each integrated system and for its application to each model, targeted specification document called PROXI programming targeted specification is issued, which will describe the format of coding the proxi file.

It comprises a binary file in which each single bit has a meaning consistent with the format defined by the specification.



PROXI

CRC Check

In order to avoid mismatches between the nodes in the car, is required implementation of a CRC check.

During the generation or modification of the PROXI file, the tool must check from byte 26 to the end of the PROXI file based on a polynomial calculation of CRC-16.

The result must be written inside the configuration code.

The nodes that receive the proxci must:

- Calculate the CRC from byte 26 to the end of proxci file
- Compare the result with the values in the configuration code
- Accept the proxci file in case of positive result of the comparison or refuse it in case of negative results of the comparison

Byte1*	Byte2*	Byte3*	Byte4*	Byte5*	Byte6*	Byte7*	Byte8*	Byte9*	Byte10*	Byte11*
						CRC	CRC	CRC	CRC	CRC



PROXI

Check configuration

The master node is requested to analyze the integrity and coherence of the configuration of each control unit involved, and to collect data regarding the system status.

The procedure to carry out this task is called check configuration, and involves all the nodes that take part in proxy procedure.

The master node sends the **CFG_DATA_CODE_REQUEST** message to the network, transmitting its own configuration code.

After receiving the CFG data request, all the nodes that take part to the proxy will send **CFG_DATA_CODE_RSP_XXX** message.

The master will analyze the content, and if it is equal to the one stored, then the check configuration will be OK.

Configuration check is performed at every transition to key ON.



PROXI

Check configuration

The check configuration can fail if:

- One or more nodes have not responded with its own coded CFG_DATA_CODE_RSP_XXX
- Unexpected responses are received from one or more nodes (nodes not involved in proxy)
- Value of EOL signal not the expected one
- Configuration codes received are different to its own stored code

If the check fails , then the master node may repeat the cycle 3 times before to raise a fail.



PROXI

Check configuration fail snapshot AA

It is the copy of \$40 A2 END OF LINE STATUS , carried out in case of a negative configuration check.

The RDI is \$40 AA.

The information contained must be preset to 0 and be reset whenever the configuration check fails and reset when a new configuration check ends positive.

PROXI

System Diagnosis snapshot A1

The system diagnosis function is used to synthesizing inside the Can master node a certain number of diagnostic information, used with database purpose.

The main data are related to the presence of fault in the ECU, present or stored in memory.



PROXI

EOL status snapshot A2

The snapshot contains information on system status after each CONFIGURATION CHECK.

The service to read it is the RDI **\$40 A2**

In detail, the snapshot contains the following information:

- Config_CAN_System: nodes configured in the proxo file, as present on CAN Bus
- Config_EOL_CAN_System: nodes configured in the PROXI file as participant
- Active can systems: nodes active on network
- EOL_CAN_system: nodes that take part in proxo procedure, update their network management message with EOL=1
- EOL_OK_CAN: nodes with EOL=1, that correctly responds to check configuration.

Not all the nodes configured as present, must take part to the proxo procedure.



PROXI

Feedback for PROXI writing KO

The Nodes receiving the PROXI file must check its content. This procedure, through controls coherent with the diagnostic protocol foreseen, can end with a positive or negative outcome.

Basically the reason of the reject are: file not conforming with the requirements, file not coherent with the nature of the node receiving or CRC calculation not OK.

The type of check for PROXI file are:

- **Configuration Code format check** □ check configuration procedure KO
- **Null format check** □ ASCII strings are aligned to the left and if empty spaces have to be filled, the characters are null (0x20)
- **Date format check** □ the format is YYYY/MM/DD
- **Hardware compatibility check** □ nodes relevant to information included in this field must carry out a coherence check with their own hw (i.e. proximeter temperature sensor present, but hw of the Ecu not populated)
- **Invalid value check** □ files containing information defined as invalid shall be rejected
- **Proxim compatibility check** □ coherence information shall respect coherence tables, i.e. proximeter table requires dependences of two or more parameters.



PROXI

UDS Service

The proxy is a stored parameter, both for the master and all nodes taking part in proxy procedure.

To read it the service used is **\$22**.

The parameters to read are:

20 24 □ CAR proxy data, stored by the master node

20 23 □ ECU proxy data, contains the proxy data read by the single ECU.

To write it the service is **\$2E**



ECU SW Download



ECU sw Download – Electronic systems re-programming

Re-programming is the operation allowing diagnostic instrument to send informations to the system, which receives, controls and stores them.

An ECU can be re-programmed, by flashing on it a new sw.

The main operations performed by the diagnostic tool during the download are:

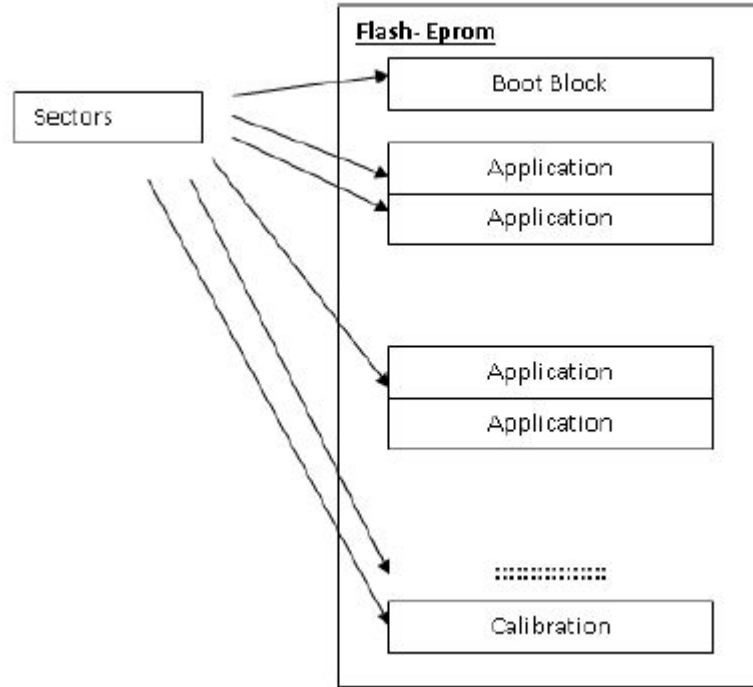
- **Erase the EEPROM**
- **Transfer data to the ECU**



A sw is generally structured in blocks.

The main type of block are:

- Bootloader
- Application
- Calibration





Download Kit Files

- **.BIN file – data file**

Containing code and data to be downloaded.

It contains all flash EEPROM data.

- **.PRM file – parameter file**

Containing all the parameters used by protocol during reprogramming (i.e. target address, access mode, security access parameters...) and the fingerprint to update

- **.IDX file – index file**

Containing all operations necessary to match a pair of data + parameter files.

It also contains the informations necessary to check if the download is executable or not, depending on the identifications of the ECU to flash.



Download **.IDX file – data file example**

80818283

HwNumb

A6

NewSwNumb

A6A7

123456

AABBCCDDEE

NOMEFILE.BIN

20011129

OldSwNumb1

A6A6

OldSwNumb2

A6A5

OldSwNumb3

A6A3

- New sw spare part number
- Old HW number
- Old HW version
- New SW number
- New SW version
- New exhaust regulation approval number
- New Identification options
- Old SW number
- Old SW version



Download .PRM file – data file example

```
Header Format Identifier      05 = Download on C1 CAN 29 possibility to use High or Normal priority CAN IDs
Finger Print                 01 = applicationSoftware + applicationData
CAN ID Tester                18DA40F1
CAN ID ECU                   18DAF140
Source Address               F1
Target Address               40
max baudrate                 10400 baud
Access Mode                  01
type of Seed&KeyId           4F
DataFormatId                 00 = Compression Method none; Encrypting Method none
address&Size                  33 = memoryAddress length 3 bytes; memorySize length 3 bytes
Calculate checksum method    01 = only after all segments
Number of segment to download 1
Checksum whole Flash start Address 00010000
Checksum whole Flash stop Address 000EC800
Checksum whole Flash expected checksum 3A1A
Segment: 1
First byte address to be read from file 00000000
Number of bytes to be read from file 000EC800
start address erase flash 00010000
stop address erase flash 000EC800
memory Address 00010000
UnComprMemorySize 00000800
checksum start Address 00010000
checksum stop Address 000EC800
expected checksum 3A1A
```



Download

Preliminary operations

Before to download the sw, the tool performs some preliminary operations to check if the flashing is available.

During operation check, the whole identification block of the ECU must be available.

For example if the HW is not compatible with the one in the .idx file, then the download won't start, to avoid problems on the ECU.

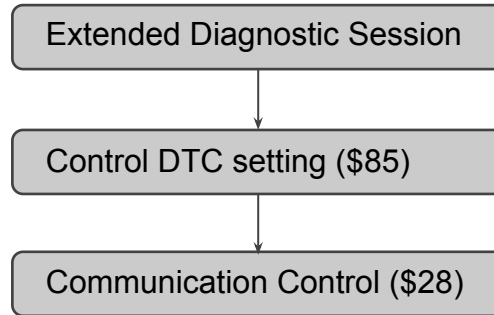


Download

Data transfer flow – Tester to all functional addressing

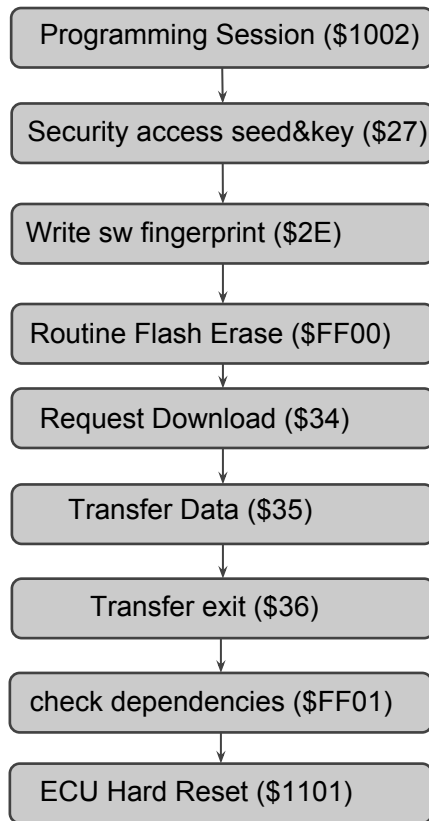
Before to start a download procedure, the tester sends in functional addressing, some requests to all the ECU present on the same can network.

During a download a minimum level of busload must be guaranteed.





Download Data transfer flow – Tester to ECU





Download

Data transfer flow – End of flashing sequence

Once the flashing procedure has been performed, new identifications will be available on ECU electronic label, with updated fingerprint (depending on the type of we downloaded: boot, application, calibration)

After the download ended, the system must update the following data, at the following Key On:

- Number of flash re-programming
- Odometer content to the last F-ROM update



Addendum: Acronyms



Diagnostic – Definitions, acronyms and abbreviation

- **DTC:** Diagnostic Trouble Code
- **RDI:** Read Data by Identifier
- **ID:** Identification
- **IO control:** Input Output control
- **CDD:** Candela Diagnostic Data
- **PROXI:** Programming and configuration of integrated systems
- **EOL:** End Of Line
- **DID:** Data Identifier
- **UDS:** Unified Diagnostic Service

CONTACTS

Bruno D'Angelo
Bruno.Dangelo@kineton.it

Pasqualina Verolino
Pasqualina.Verolino@kineton.it

Lorenzo Aurilio
Lorenzo.Aurilio@kineton.it
Pasquale Perrotta
Pasquale.Perrotta@kineton.it

Via G. Porzio,
Centro Direzionale Isola B3
80143 Napoli Via G. Porzio

www.kineton.it

